

REDSPHER

Full Stack Developer

Interjúfelkészítő kézikönyv

TypeScript • React • Node.js • AWS • REST APIs
CI/CD • rendszertervezés • LLM • MCP • AI agents

Telefonon jól olvasható, egyoszlopos kiadás
Magyar magyarázatok, angol szakkifejezésekkel
2026. július

Hogyan használd ezt az anyagot

A cél nem a definíciók bemagolása, hanem rövid, strukturált és hiteles interjúválaszok kialakítása.

Prioritások

Az ESSENTIAL kérdéseket biztosan gyakorold. A VALÓSZÍNŰ kérdések gyakran előkerülnek egy hasonló full-stack interjún. A BÓNUSZ témák akkor fontosak, ha mélyebbre mennek vagy AI/architecture irányba fordul a beszélgetés.

- Első kör: csak az ESSENTIAL kártyákat olvasd végig.
- Második kör: válaszolj hangosan, 60-90 másodpercben.
- Harmadik kör: minden válaszhoz köss egy saját projektpéldát.
- A system design résznél mindig mondd ki a feltételezéseidet és a trade-offokat.
- A konkrét teljesítményszámokat csak akkor használd, ha vállalod és meg tudod indokolni őket.

Ajánlott válaszformula

Definíció egy mondatban → mikor használd → fontos trade-off → rövid saját példa. Ez technikai és üzleti kérdéseknél is jól működik.

Tartalom

Hogyan használd ezt az anyagot	2
1. Mit keres valójában a Redspher?	4
2. TypeScript	7
3. React	13
4. Node.js és backend	20
5. REST API-k és adatkezelés	27
6. AWS és cloud architecture	32
7. Git, CI/CD, minőség és observability	38
8. LLM, MCP és AI agents	42
9. System design gyakorlatok	49
10. Business és viselkedési kérdések	55
11. Saját történetbank	60
12. Gyors kérdések - egymondatos válaszok	64
13. 30 perces próbainterjú	68
14. Utolsó napi cheat sheet	70
15. Források és hatókör	72

1. Mit keres valójában a Redspher?

A hirdetés alapján ez nem tisztán frontend szerep. End-to-end feature ownership, pragmatikus üzleti gondolkodás, AWS és AI-kísérletezés együtt jelenik meg.

A legvalószínűbb fókusz

Erős React + TypeScript alapok; Node.js back-end és REST API-k; AWS-ben működő rendszerek; architekturális döntések; stakeholder kommunikáció; AI agents és MCP iránti gyakorlati érdeklődés; legacy PHP integráció mint lehetséges plusz.

A hirdetésből levezethető interjústruktúra

- Rövid bemutatkozás és korábbi full-stack projektek.
- React/TypeScript mélyítés, különösen state, effects, komponenshatárok és type safety.
- Node.js, REST, hibakezelés, skálázás és adatfolyamok.
- AWS architektúra: compute, adatbázis, messaging, IAM, monitoring és deployment.
- Pragmatikus system design egy logisztikai vagy workflow problémára.
- AI/LLM/MCP beszélgetés, valószínűleg koncepcionális és prototípus-szintű mélységben.

- Business fit: hogyan tisztázod az igényeket és hogyan választasz egyszerű, fenntartható megoldást.

A legerősebb pozicionálásod

Senior full-stack fejlesztő vagy, erős React/TypeScript és valós idejű Node.js háttérrel. Emeld ki az end-to-end felelősséget, a release és observability tapasztalatot, az automatizálást, valamint azt, hogy az AWS és AI területet aktívan bővíted.

60 másodperces bemutatkozás - angolul

Suggested answer

I am a full-stack developer with more than fifteen years of software development experience, with my strongest recent focus on React, TypeScript and Node.js. I have built and maintained user-facing applications as well as scalable real-time backend services, and I have worked across the full delivery cycle from architecture and implementation to release, monitoring and production troubleshooting. I also enjoy automation and practical AI use cases, and I have recently been deepening my AWS knowledge. What interests me in this role is the combination of end-to-end product development, business collaboration and the opportunity to prototype AI-driven workflows in a real logistics environment.

Miért jó ez?

Nem technológialistát mond, hanem szerepet

és értéket: full-stack ownership, skálázhatóság, production responsibility, automation, AWS és AI. Ez pontosan a hirdetés nyelvéhez kapcsolódik.

Miért érdekel a szerep? - angolul

Suggested answer

The role matches both my experience and the direction in which I want to grow. I can contribute immediately with React, TypeScript and Node.js, while the AWS and AI-agent parts are areas where I am already investing significant time. I also like that the team is business-oriented. I prefer solving a real operational problem with a pragmatic solution over introducing complexity only because a technology is fashionable.

2. TypeScript

A hirdetés külön kiemeli a strong TypeScript tudást, ezért itt nem elég az, hogy „JavaScript with types”. A döntések és trade-offok számítanak.

ESSENTIAL

Mi a TypeScript legfontosabb értéke egy nagyobb rendszerben?

A TypeScript statikus típusellenőrzést és jobb fejlesztői visszajelzést ad a JavaScript fölé. A legnagyobb értéke nem az, hogy minden hibát megszüntet, hanem hogy a szerződéseket láthatóvá és géppel ellenőrizhetővé teszi komponensek, modulok és API-határok között. Ez javítja a refaktorálhatóságot, az IDE támogatását és a csapaton belüli kommunikációt. A runtime adatokat ettől még validálni kell.

Mit keres az interjúztató? Érti-e, hogy a type system fejlesztési időben működik, és nem helyettesíti a runtime validationt.

Saját példád A React/Node kódbázisokban a típusokkal biztonságosabban tudtál API-szerződéseket és komponenspropokat módosítani.

Buktató Ne mondd, hogy a TypeScript garantálja a runtime helyességet.

ESSENTIAL

Mi a különbség az any és az unknown között?

Az `any` gyakorlatilag kikapcsolja a típusellenőrzést az adott értéken. Az `unknown` azt mondja, hogy az érték típusa még nem ismert, ezért használat előtt `narrowing` vagy `validation` szükséges.

Külső inputnál, JSON parsing után vagy hibakezelésnél az `unknown` biztonságosabb alapértelmezés.

Mit keres az interjúztató? Biztonságos határkezelés és tudatos típusválasztás.

Saját példád API response vagy caught error feldolgozásánál előbb ellenőrzöd a shape-et, és csak utána használod.

Buktató Az `as type assertion` ne legyen a validation helyettesítője.

ESSENTIAL

Mikor használnál union type-ot és discriminated unionot?

Union type-pal több lehetséges állapotot modellezünk. Discriminated union esetén minden változatnak van egy közös, literál típusú megkülönböztető mezője, például `status` vagy `type`.

Ez különösen jó async UI state, domain eventek és API eredmények modellezésére, mert a switch ágakban pontos `narrowing`-et kapunk, és `never` segítségével `exhaustiveness check` is készíthető.

Mit keres az interjúztató? Modellezi-e az állapotokat úgy, hogy lehetetlen kombinációk ne jöhessenek létre.

Saját példád Loading/success/error UI state vagy különböző websocket message típusok.

Buktató Ne használj sok egymástól független booleant ugyanazon állapot leírására.

VALÓSZÍNŰ**Mi az interface és a type alias közötti különbség?**

Mindkettő használható objektum shape-ek leírására. Az `interface` természetesen bővíthető és támogatja a declaration merginget. A `type` rugalmasabb: union, intersection, tuple és más összetett típusok is kifejezhetők vele.

A fontosabb szempont a projektkonzisztencia és az, hogy a választás illeszkedjen a modellezési igényhez.

Mit keres az interjúztató? Nem dogmatikusan választ-e, hanem érti a tényleges különbséget.

Buktató Ne állítsd, hogy az egyik általánosan „jobb” a másiknál.

ESSENTIAL**Mire valók a generics?**

A generics lehetővé teszi, hogy újr felhasználható, mégis típusbiztos komponenseket és függvényeket írjunk. A típuskapcsolatot őrzi meg a bemenet és a kimenet között, ahelyett hogy `any`-re váltanánk.

Constraints segítségével megadhatjuk, milyen képességeket várunk el a típustól, például hogy legyen `id` mezője.

Mit keres az interjúztató? Tud-e újr felhasználható API-t tervezni típusinformáció elvesztése nélkül.

Saját példád Generic API client, reusable table vagy typed event handler.

Buktató A túl bonyolult generics ronthatja az olvashatóságot; csak valódi kapcsolat modellezésére használd.

VALÓSZÍNŰ**Mi a structural typing?**

A TypeScript alapvetően a típus szerkezetét hasonlítja, nem a deklarált nevét. Ha egy érték rendelkezik a szükséges mezőkkel és azok kompatibilis típusúak, akkor hozzárendelhető.

Ez jól illeszkedik a JavaScript objektummodelljéhez, de API-határokon néha nominal-like megoldás, például branded type segíthet az összekeverhető azonosítóknál.

Mit keres az interjúztató? Érti-e, miért kompatibilisek külön deklarált, azonos shape-ű objektumok.

ESSENTIAL**Mit jelent a type narrowing?**

A narrowing során egy szélesebb típust runtime feltétel alapján szűkítünk. Ilyen lehet a `typeof`, `instanceof`, `in`, equality check, discriminant mező vagy saját type guard.

A jó narrowing a program vezérlési logikájával együtt bizonyítja a compilernek, hogy az adott művelet biztonságos.

Mit keres az interjúztató? Nem assertionökkel kényszeríti-e a compilert, hanem bizonyítja a típust.

Buktató A custom type guard csak akkor megbízható, ha a runtime ellenőrzés tényleg teljes.

VALÓSZÍNŰ**Mire jó a `never` típus?**

A `never` olyan értéket jelöl, amely nem fordulhat elő. Lehet egy mindig hibát dobó függvény visszatérési típusa, és használható exhaustiveness checkre.

Egy discriminated union switch végén az ismeretlen ág `never`-re kényszerítése compile-time jelzi, ha új variánst adtunk hozzá, de nem kezeltük.

Mit keres az interjúztató? Gondol-e a jövőbeli változtatások biztonságára.

ESSENTIAL

Miért fontos a `strict` mód?

A `strict` több szigorú compiler opciót kapcsol be, köztük a nullability, implicit any és function variance ellenőrzéseket. Ezek több hibát hoznak elő fejlesztési időben, és tisztább szerződéseket kényszerítenek ki.

Legacy rendszerben fokozatos bevezetés lehet szükséges, de új kódbázisnál jó alapértelmezés.

Mit keres az interjúztató? Tud-e tudatosan minőségi alapbeállításokat választani.

Saját példád Legacy modulok migrációjánál fokozatosan lehet szigorítani, különösen a külső határoknál.

Buktató Ne kapcsolod ki globálisan a szigorúságot egy lokális probléma miatt.

VALÓSZÍNŰ

Hogyan kezelnéd a külső API-ból érkező adatokat?

A TypeScript típusdeklaráció önmagában nem validálja a response-t. A hálózati határon runtime schema validationt használnék, majd a validált adatot alakítanám domain típusúvá.

Emellett külön kezelném a transport hibát, az invalid response-t és a business-level hibát.

Mit keres az interjúztató? Érti-e a compile-time és runtime biztonság határát.

Saját példád REST/GraphQL integrációkban a bejövő adatok normalizálása és a hibák megfigyelhető kezelése.

Buktató A `response as MyType` nem validáció.

BÓNUSZ

Mikor használnál branded type-ot?

Ha két primitív reprezentáció azonos, de üzletileg nem felcserélhető, például `ShipmentId` és `UserId`, branded type-pal megakadályozható a véletlen keverés.

Ezt érdemes csak fontos domain-határokon alkalmazni, mert plusz komplexitást ad.

Mit keres az interjúztató? Domain modellezési érzék és a „make invalid states unrepresentable” gondolkodás.

VALÓSZÍNŰ

Hogyan szerveznéd a típusokat frontend és backend között?

A közös szerződések megoszthatók egy külön package-ben vagy generálhatók OpenAPI/GraphQL schema alapján. A domain modelleket viszont nem mindig célszerű egy az egyben megosztani, mert a frontend view model és a backend domain model eltérhet.

A cél a drift csökkentése, nem a rétegek mester-séges összekapcsolása.

Mit keres az interjúztató? Szerződéskezelés, coupling és verziózás közötti trade-off.

Saját példád Monorepo vagy generált API kliens jó példa lehet.

Buktató Ne ossz meg adatbázis-entitásokat közvetlenül a UI-val.

3. React

A válaszokban különítsd el a render logikát, az event handlert és az external system szinkronizációt. Ez a modern React gondolkodás egyik kulcsa.

ESSENTIAL

Mi történik egy React render során?

A komponensfüggvény lefut, és az aktuális props/state alapján leírja a következő UI-t. A rendernek tisztának kell lennie: ugyanazon inputra ugyanazt az outputot adja, és nem végez külső side effectet.

A commit fázisban a React alkalmazza a szükséges DOM változtatásokat, majd az effects futnak.

Mit keres az interjúztató? Érti-e a render és a side effect közötti határt.

Buktató Ne úgy magyarázd, mintha minden render teljes DOM újraépítés lenne.

ESSENTIAL

Mire való a `useEffect`, és mire nem?

A `useEffect` arra való, hogy a komponenst külső rendszerrel szinkronizáljuk: subscription, timer, browser API vagy imperative library. Nem szükséges olyan érték kiszámítására, amely render közben a props/state alapján meghatározható. Az effectnek legyen helyes dependency listája és cleanupja.

Mit keres az interjúztató? Elkerüli-e az effect-alapú adatfolyamot, ha egyszerűbb deriváció is elég.

Saját példád WebSocket subscription, analytics listener vagy külső widget integráció.

Buktató Az üres dependency array nem automatikus megoldás; stale closure-t és hibás szinkronizációt okozhat.

ESSENTIAL

Mi a state colocation, és miért fontos?

A state-et a lehető legalacsonyabb közös komponensben tartjuk, ahol minden fogyasztó eléri. Így kisebb a re-render hatókör, kevesebb a coupling és egyszerűbb az adatáramlás.

Csak akkor emelném fel vagy tenném globálissá, ha több távoli résznek valóban közös, tartós állapotra van szüksége.

Mit keres az interjúztató? Nem visz-e mindent reflexből globális store-ba.

Saját példád Nagy React alkalmazásban feature-szintű state és jól definiált globális state határ.

ESSENTIAL

Mi a különbség a controlled és uncontrolled input között?

Controlled inputnál az értéket React state vezérli, és minden változás eseményen keresztül frissíti azt. Uncontrolled esetben a DOM tartja az aktuális értéket, amelyet ref-fel vagy submitkor olvasunk ki.

Controlled megoldás jobb komplex validációhoz és dinamikus UI-hoz; uncontrolled egyszerűbb és gyakran hatékonyabb nagy formoknál.

Mit keres az interjúztató? Tud-e form architecture trade-offot megfogalmazni.

ESSENTIAL

Hogyan kerülöd el a fölösleges re-rendereket?

Először mérnék React Profilerrel, nem vakon optimalizálnék. Ezután state colocation, stabil komponenshatárok, megfelelő list key-k, valamint szükség esetén memoization jöhet szóba.

`React.memo`, `useMemo` és `useCallback` csak akkor hasznos, ha a referenciastabilitás ténylegesen megszüntet drága munkát vagy child renderelést.

Mit keres az interjúztató? Mérésvezérelt-e, és érti-e a memoization költségét.

Saját példád Nagy listák, dashboardok vagy sok adatot megjelenítő SPA optimalizálása.

Buktató A memoization nem általános „best practice”; hibásan használva csak bonyolít.

VALÓSZÍNŰ

Miért fontos a listák `key` propja?

A `key` segít a Reactnek az elemek identitását követni két render között. Stabil, egyedi domain azonosító kell, különösen beszúrás, törlés és rendezés esetén.

Az `index` `key` csak statikus listán elfogadható, ahol a sorrend és az elemek nem változnak.

Mit keres az interjúztató? Érti-e, hogy a `key` az állapot és az identitás megőrzésére is hat.

Buktató Instabil vagy random `key` minden rendernél remountot okozhat.

ESSENTIAL**Hogyan kezelnél async adat-betöltést?**

Külön modellezném a loading, success, empty és error állapotot. Gondolnék cancellationre vagy stale response problémára, cache-re, retryra és deduplicationre.

Komplex alkalmazásban célszerű server-state libraryt használni ahelyett, hogy minden fetch lifecycle-t egyedi effectsorozattal implementálnánk.

Mit keres az interjúztató? Helyes state modellezés és race condition kezelés.

Saját példád REST/GraphQL feature-eknél loading/error UX és monitoring.

Buktató Ne nyeld el a hibát, és ne hagyd a UI-t végtelen loading állapotban.

VALÓSZÍNŰ**Mikor használnál Contextet, és mikor külön state managementet?**

Context jó ritkán változó, széles körben szükséges értékekhez, példáulul theme, locale vagy dependency injection. Gyakran változó nagy állapotnál önmagában könnyen széles re-rendereket és nehéz szervezhetőséget okoz. Külön store akkor indokolt, ha összetett globális state, fejlett debugging, middleware vagy sok cross-feature interaction van.

Mit keres az interjúztató? Technológia helyett probléma alapján választ-e.

Saját példád Redux/Redux-Saga tapasztalatod mellett hangsúlyozhatod, hogy nem minden feature igényel globális store-t.

ESSENTIAL**Mi a stale closure probléma?**

A JavaScript closure a render pillanatában látható értékeket zárja be. Async callback vagy effect később régi state-et használhat, ha a dependency lista hibás vagy a frissítés nem funkcionális.

Megoldás lehet helyes dependency, functional state update, ref a nem-renderelt mutábilis értékhez, vagy az adatfolyam újratervezése.

Mit keres az interjúztató? Érti-e az effects és async callbackek gyakori hibáját.

Saját példád WebSocket event handler vagy timer esetén tipikus.

Buktató A dependency kikapcsolása lint commenttel gyakran csak elrejti a problémát.

VALÓSZÍNŰ**Mire jó az Error Boundary?**

Az Error Boundary a descendant render, lifecycle és constructor hibákat fogja meg, és fallback UI-t mutathat. Nem fogja meg automatikusan az event handler, async callback vagy server-side hibákat.

Feature-szintű boundary jó hibalokalizációt és jobb felhasználói élményt ad.

Mit keres az interjúztató? Hibaisolációban és recoverabilityben gondolkodik-e.

Saját példád Sentry integráció mellett boundaryból logolható a komponenshiba és megjeleníthető célzott fallback.

VALÓSZÍNŰ**Hogyan tervezel jól újrafelhasználható komponenst?**

Először a konkrét use case-eket és az invariáns részt keresem. Kis, világos API-t adok, compositiont preferálok túl sok boolean prop helyett, és különválasztom a domain logikát a prezentációtól.

Accessibility, loading/error állapot és testability is része a komponens API-jának.

Mit keres az interjúztató? Nem absztrahál-e túl korán, és tud-e stabil komponensszerződést tervezni.

Saját példád Storybook és Figma együttműködésed releváns példa.

Buktató A „universal component” sok konfigurációval gyakran rosszabb, mint néhány célzott komponens.

VALÓSZÍNŰ

Hogyan tesztelnél egy React feature-t?

A kritikus viselkedést a felhasználó szemszögéből tesztelném: render, interakció, validáció, loading/error és eredmény. A tiszta logikát unit tesztekkel, a komponensintegrációt Testing Library szemlélettel, a fő üzleti folyamatot kevés E2E teszttel fedném.

Nem a belső state-et vagy implementation detailt tesztelném.

Mit keres az interjúztató? Fenntartható teszt-piramis és viselkedésalapú tesztelés.

Saját példád Jest és Selenium tapasztalatod kapcsolható ide.

BÓNUSZ

Mit jelent a hydration, és mikor lehet probléma?

SSR esetén a szerver HTML-t küld, majd a kliens JavaScript hozzákapcsolja az interaktivitást. Hydration mismatch akkor lehet, ha a szerver és a kliens első renderje eltér, például idő, random érték vagy böngésző-only adat miatt.

A megoldás determinisztikus első render és a kliensspecifikus logika megfelelő elkülönítése.

Mit keres az interjúztató? Van-e modern web rendering alapismerete.

Saját példád Next.js alapismeretedet említheted, de ne állíts mély production tapasztalatot, ha nem volt.

VALÓSZÍNŰ

Hogyan közelítenél egy komplex üzleti formhoz?

A domain szabályokat elválasztanám a megjelenítéstől, schema-alapú validációval és jól modellezett form state-tel. Kezelném a mezőfüggőségeket, draft mentést, server validationt, accessibilityt és a részleges hibákat.

Nagy formnál fontos a teljesítmény és az, hogy ne minden mezőváltozás renderelje újra az egész oldalt.

Mit keres az interjúztató? Komplex end-to-end feature tervezés.

Buktató A kliensoldali validáció nem helyettesíti a backend validációt.

4. Node.js és backend

A Redspher core logistics alkalmazásokat említi. A backend válaszokban hangsúlyozd a hibahatárokat, idempotenciát, observabilityt és a skálázási modellt.

ESSENTIAL

Hogyan működik a Node.js event loop?

A JavaScript kód alapvetően egy event loop threaden fut, miközben sok I/O műveletet az operációs rendszer vagy a libuv háttérmechanizmusai kezelnek. Amikor az eredmény elkészül, callback vagy promise continuation kerül a megfelelő queue-ba.

Ezért Node.js erős I/O-bound workloadnál, de CPU-intenzív munka blokkolhatja az event loopot.

Mit keres az interjúztató? Érti-e, miért tud sok kapcsolatot kezelni és mikor romlik a latency.

Saját példád Nagy számú WebSocket kapcsolat kezelése és PM2 clustering jó saját példa.

Buktató Ne mondd egyszerűen, hogy „Node single-threaded”; a runtime és az I/O végrehajtás ennél összetettebb.

ESSENTIAL

Mit tennél CPU-intenzív feladattal Node.js-ben?

Először mérném, valóban CPU bottleneck-e. Ezután worker thread, külön process/service,

queue-alapú background job vagy célzottan más runtime jöhet szóba.

A request path-on kerülni kell a hosszú szinkron számítást, mert minden más kérés latenciáját rontja.

Mit keres az interjúztató? Tud-e workload alapján architektúrát választani.

ESSENTIAL

Hogyan strukturálnál egy Node.js szolgáltatást?

Elválasztanám a transport réteget, az application/use-case logikát, a domain szabályokat és az infrastruktúra adaptereket. A route handler legyen vékony: parse, auth, validation, use-case hívás, response mapping. Nem a mappanév a lényeg, hanem hogy a business logic ne függjön közvetlenül frameworktől vagy adatbázistól.

Mit keres az interjúztató? Separation of concerns és tesztelhetőség.

Saját példád A korábbi backend modernizációkban a protocol, auth és scaling felelősségek szétválasztása.

Buktató Ne alakíts ki túl sok absztrakciót egy kicsi CRUD service-ben.

ESSENTIAL

Hogyan kezelnéd a hibákat egy backendben?

Különválasztanám a validation, authentication, authorization, domain, dependency és unexpected hibákat. Központi error mapping egységes API response-t és megfelelő HTTP státuszt adna.

Minden hiba kapna correlation ID-t és strukturált logot; az expected hibák nem feltétlenül 500-asok, az unexpected hibák pedig nem szivárogtathatnak belső részleteket.

Mit keres az interjúztató? Megbízható error semantics, diagnosztizálhatóság és security.

Saját példád Hibakezelési és observability tapasztalat Sentry/Kibana/Prometheus környezetből.

Buktató A catch-and-log-and-rethrow többszörös zajt okozhat; legyen egyértelmű ownership.

ESSENTIAL

Mi az idempotencia, és miért fontos?

Egy művelet idempotens, ha ugyanazon kérés ismétlése nem okoz újabb üzleti hatást. Elosztott rendszerekben retry, timeout és duplicate message miatt alapvető.

Megoldás lehet idempotency key, egyedi constraint, deduplication store vagy állapotgép, amely felismeri a már feldolgozott műveletet.

Mit keres az interjúztató? Gondol-e retry és partial failure helyzetekre.

Saját példád Payment helyett logisztikai booking vagy shipment creation jó példa.

Buktató A HTTP method elméleti idempotenciája nem garantálja automatikusan az implementáció idempotenciáját.

VALÓSZÍNŰ

Hogyan skáláznál egy stateful WebSocket szolgáltatást?

A connection state lokálisan maradhat, de a cross-instance eseményeket pub/sub vagy broker teríti. Load balancer, connection health, backpressure, authentication refresh és graceful draining szükséges.

Ha session affinity van, akkor is úgy terveznék, hogy instance kiesés kezelhető legyen, és a fontos állapot tartós vagy újraépíthető legyen.

Mit keres az interjúztató? Valódi production scaling tapasztalat.

Saját példád Redis pub/subbal szinkronizált, horizontálisan skálázott support chat és nagy concurrent connection szám.

Buktató Ne támaszkodj kizárólag in-memory state-re üzletileg fontos adatoknál.

ESSENTIAL

Mit jelent a graceful shutdown?

Az alkalmazás leállításkor nem fogad új munkát, de időt ad a folyamatban lévő kérések, queue message-ek és kapcsolatok biztonságos befejezésére. Lezárja az adatbázis poolokat, flusholja a telemetryt, majd határidő után kilép. Deployment és autoscaling során ez csökkenti az elveszett munkát és a hibás válaszokat.

Mit keres az interjúztató? Production readiness és lifecycle gondolkodás.

Saját példád Rolling update és PM2 clustering tapasztalatod ide kapcsolódik.

VALÓSZÍNŰ

Mi a backpressure?

Backpressure akkor szükséges, amikor a producer gyorsabban termel, mint ahogy a consumer fel tudja dolgozni. Ilyenkor korlátozni,

bufferelni, lassítani vagy elutasítani kell a munkát.

Node stream-ek, message queue-k és Web-Socket outbound buffer esetén is fontos, különben memóriefelhalmozódás és latency explosion lesz.

Mit keres az interjúztató? Érti-e a throughput és a stabilitás kapcsolatát.

Saját példád Nagy forgalmú realtime rendszereknél releváns.

VALÓSZÍNŰ

Mikor használnál queue-t szinkron API-hívás helyett?

Ha a feladat hosszabb, retryzható, nem igényel azonnali választ, vagy lazítani akarom a producer és consumer couplingját. Queue-val kisimítható a terhelés és izolálhatók a dependency hibák.

Cserébe eventual consistency, duplicate delivery, ordering és observability kérdéseket kell kezelni.

Mit keres az interjúztató? Trade-offokat lát-e, nem csak a queue előnyeit.

Saját példád Report generation, notification vagy AI enrichment háttérfolyamat.

Buktató Queue nem oldja meg magától az idempotenciát.

ESSENTIAL

Hogyan validálnád egy API requestjét?

A transport határon runtime schema validationt használnék. Külön kezelném a szintaktikai shape-et és a business invariantokat: például

a dátum formátuma schema kérdés, az hogy a pickup korábbi legyen a deliverynél domain szabály.

A validált adatot explicit command vagy DTO formára alakítanám.

Mit keres az interjúztató? Határkezelés és domain logika elkülönítése.

Buktató Ne bízz a TypeScript típusban hálózati inputnál.

VALÓSZÍNŰ

Hogyan védenél egy Node.js API-t?

Alapok: input validation, authentication és authorization szétválasztása, least privilege, secret management, rate limiting, timeout, dependency frissítés, biztonságos header és strukturált audit log.

Emellett védeném az adatbázist injectiontól, korlátoznám a response részletességét, és külön figyelnék SSRF-re, ha a rendszer külső URL-eket hívhat.

Mit keres az interjúztató? Security-by-default és threat awareness.

Buktató A JWT megléte önmagában nem jelent teljes API securityt.

BÓNUSZ

Mikor használnál streamet?

Nagy vagy folyamatos adat feldolgozásánál, amikor nem akarjuk az egészet memóriába tölteni. A stream lehetővé teszi a chunk-alapú feldolgozást és a backpressure kezelését.

Példák: nagy export, file upload, proxyzás, log processing.

Mit keres az interjúztató? Memória-hatékony I/O gondolkodás.

5. REST API-k és adatkezelés

A jó válaszok az HTTP semantics, domain model, kompatibilitás és üzleti következmények együttese köré épülnek.

ESSENTIAL

Mitől REST egy API?

A REST egy architectural style. A fontos elvek közé tartozik a resource-oriented model, egységes interface, stateless requestek, cache-elhetőség, rétegezhetőség és a reprezentációkon keresztüli állapotátmenet.

Egy JSON over HTTP API nem automatikusan RESTful; a lényeg a következetes resource és HTTP semantics.

Mit keres az interjúztató? Nem csak endpoint-szintű definíciót tud-e.

Buktató Ne ragadj le annál, hogy „GET, POST, PUT, DELETE”.

ESSENTIAL

PUT és PATCH között mi a különbség?

A PUT általában egy resource teljes reprezentációjának létrehozását vagy cseréjét jelenti, és idempotens. A PATCH részleges módosítást ír le, amelynek idempotenciája az alkalmazott patch semantics-től függ.

A választást a szerződésben egyértelművé kell tenni, különösen hiányzó mezők és null értékek esetén.

Mit keres az interjúztató? HTTP semantics és világos contract.

ESSENTIAL

Hogyan terveznél hibaválasz-formátumot?

Legyen konzisztens gépi kód, emberi üzenet, correlation ID és szükség esetén field-level details. Ne szivárogtasson stack trace-t vagy belső adatot.

A kliens tudjon különbséget tenni validation, auth, conflict, rate limit és transient dependency hiba között.

Mit keres az interjúztató? Kliensoldali kezelhetőség és operációs diagnosztika.

Saját példád Frontend és backend oldalt ismerve meg tudod indokolni, miért fontos a stabil error contract.

ESSENTIAL

Hogyan verzióznál egy API-t?

Először kompatibilis evolúcióra törekednék: új opcionális mezők, tolerant reader és deprecation. Breaking change esetén explicit verzió lehet URL-ben, headerben vagy media type-ban; a választásnál egyszerűség és tooling számít.

A verziózás mellett kell migration plan, usage telemetry és sunset kommunikáció.

Mit keres az interjúztató? Nem csak a `v1` prefixet említi-e, hanem lifecycle-t is.

Buktató Új enum érték is lehet breaking change, ha a kliens exhaustive módon kezeli.

VALÓSZÍNŰ**Offset vagy cursor pagination?**

Offset pagination egyszerű és jól működik kis, stabil adathalmazon, de nagy offsetnél lassú lehet és concurrent változásoknál duplikációt vagy kihagyást okozhat. Cursor pagination stabil rendezési kulcsot használ, skálázhatóbb és konzisztens görgetést ad.

A cursor legyen opaque, és egyértelmű ordering szükséges.

Mit keres az interjúztató? Adatméret és konzisztencia alapján választ-e.

ESSENTIAL**Mikor választanál SQL adatbázist?**

Ha erős relációk, tranzakciók, összetett lekérdezések és konzisztencia fontos. Logisztikai core adatoknál, például shipment, booking, pricing rule és invoice kapcsolatainál gyakran természetes választás.

A sématervezés, indexek és query pattern együtt számítanak.

Mit keres az interjúztató? Nem technológiai divat, hanem access pattern és invariant alapján dönt-e.

Saját példád Korábbi SQL tapasztalatodat és az AWS RDS/Aurora tanulást kapcsolhatod ide.

VALÓSZÍNŰ**Mikor választanál NoSQL megoldást?**

Ha az access patternnek előre jól ismertek, nagy skála és alacsony latency kell, vagy rugalmas aggregátum-modell előnyös. Például event,

telemetry vagy nagy volumenű lookup adatoknál lehet jó.

Cserébe a kapcsolatok és ad hoc query-k nehezebbek lehetnek, ezért a modellezést a lekérdezésekből kell indítani.

Mit keres az interjúztató? Érti-e, hogy NoSQL nem „schema nélküli SQL”.

ESSENTIAL

Mi az optimistic concurrency control?

A record verzióját vagy timestampjét ellenőrizzük updatekor. Ha közben más módosította, az update nem írja felül csendben az új állapotot, hanem conflictot ad.

Jó olyan üzleti folyamatoknál, ahol ritka az ütközés, de az elveszett update elfogadhatatlan.

Mit keres az interjúztató? Concurrent üzleti módosítások és adatvesztés kezelése.

Saját példád Árképzés vagy shipment státusz módosítás jó példa.

VALÓSZÍNŰ

Mi az N+1 query probléma?

Egy kezdeti query után minden elemhez külön további query fut, így a lekérdezések száma az elemszámmal nő. Megoldás lehet join, batch loading, eager loading vagy célzott denormalizáció.

A megoldást mérni kell, mert a túl nagy join vagy túlzott eager loading is drága lehet.

Mit keres az interjúztató? Adat-hozzáférési teljesítmény és mérés.

BÓNUSZ**Hogyan közelítenél legacy PHP integrációhoz?**

Először feltérképezném a szerződést, adatownershipet, hibamódokat és release kockázatot. Stabil adapter vagy anti-corruption layer mögé tenném a legacy rendszert, és contract tesztekkel védeném az integrációt.

Fokozatos strangler megközelítést választanék big-bang rewrite helyett, ha üzletileg lehetséges.

Mit keres az interjúztató? Pragmatizmus és migration risk kezelés.

Saját példád Legacy protokoll reverse engineering és fokozatos Node.js modernizáció releváns saját példa.

Buktató Ne nevezd automatikusan rossznak a legacy rendszert; üzleti értéket hordoz és működési tudást tartalmaz.

6. AWS és cloud architecture

A hirdetés nem nevez meg konkrét AWS szolgáltatásokat. Ezért az interjúban valószínűbb az architektúrális gondolkodás, mint a ritka service-detail kérdések.

ESSENTIAL

Milyen szempontok alapján választanál Lambda és container között?

Lambda jó eseményvezérelt, rövid, változó terhelésű feladathoz, kevés infrastruktúra-menedzsmenttel. Container jobb hosszú futású, folyamatos, speciális runtime-ot vagy stabil erőforrást igénylő workloadhoz.

Vizsgálom a latencyt, cold startot, futási időt, concurrencyt, state-et, deploymentet, cost profilt és csapatkompetenciát.

Mit keres az interjúztató? Workload alapján választ-e, nem preferencia alapján.

Saját példád Serverless AWS demo projektek és container/PM2 production háttér kombinálható a válaszban.

Buktató Ne állítsd, hogy a serverless mindig olcsóbb vagy automatikusan egyszerűbb.

ESSENTIAL

Mit jelent a least privilege IAM?

Egy identity csak a feladata elvégzéséhez szükséges minimum jogosultságot kapja, a megfelelő resource-ra és actionre korlátozva.

Emberi user helyett workload role-t, rövid életű credentialt és auditálható policyt preferálnék. A jogosultságokat iteratíván szűkíteném és rendszeresen felülvizsgálnám.

Mit keres az interjúztató? Cloud security alapismeret.

Buktató A széles wildcard policy gyors prototípusban is könnyen bent marad productionben.

ESSENTIAL

Hogyan terveznél megbízható AWS workloadot?

Meghatároznám az üzleti SLO-t és failure mode-okat. Több Availability Zone, stateless compute, health check, timeout, retry backoffkal, idempotencia, queue buffering, backup és recovery test jöhet szóba.

A reliability mellett security, observability, cost és operational simplicity trade-offokat is kimondanám.

Mit keres az interjúztató? Well-Architected szemlélet és üzleti célhoz kötött döntések.

Saját példád Realtime service rolling update, monitoring és hibakezelés jó gyakorlati háttér.

Buktató A retry timeout nélkül vagy idempotencia nélkül súlyosbíthatja a hibát.

VALÓSZÍNŰ

Mikor használnál SQS-t, SNS-t és EventBridge-et?

SQS queue-alapú feldolgozásra és consumer decouplingra jó. SNS egyszerű pub/sub fan-out több subscriberhez. EventBridge event bus, routing rule-okkal és sok producer/consumer integrációval.

A választásnál delivery semantics, ordering, filtering, throughput és operational complexity számít.

Mit keres az interjúztató? Messaging pattern felismerése.

Saját példád Redis pub/sub és realtime tapasztalatod mellé mutasd, hogy tartós cloud messagingnél más guarantee-eket vársz.

ESSENTIAL

Hogyan kezelnéd a retryt?

Csak transient hibát retryznék, exponenciális backoff és jitter mellett, korlátozott próbálkozásszámmal. A teljes request deadline-t és a downstream kapacitást is figyelembe venném.

Persistent hiba esetén DLQ vagy failure workflow kell, és a műveletnek idempotensnek kell lennie.

Mit keres az interjúztató? Elosztott rendszerek stabilitása.

Buktató A kontrollálatlan retry storm tovább terhelheti a hibás dependencyt.

ESSENTIAL

Mit monitoroznál egy cloud alkalmazásban?

A felhasználói és üzleti eredményt, nem csak az infrastruktúrát. Latency, traffic, errors, saturation; emellett queue age, retry, DLQ, success rate és kritikus üzleti flow completion.

Strukturált log, metric és trace correlation ID-val; dashboard és célzott alert, amely actiont igényel.

Mit keres az interjúztató? Operability és business observability.

Saját példád Prometheus/Grafana, Sentry, Kibana és release monitoring tapasztalatod erős példa.

Buktató Túl sok zajos alert ugyanúgy kockázat, mint a kevés monitoring.

VALÓSZÍNŰ

RDS/Aurora vagy DynamoDB?

Relációs modell, tranzakció és rugalmas query esetén RDS/Aurora. Nagy skálájú, jól ismert key-based access pattern és kiszámítható alacsony latency esetén DynamoDB.

Nem csak a szolgáltatást, hanem a sémát, indexeket, consistency igényt és operation modelt választjuk.

Mit keres az interjúztató? Data access pattern alapú döntés.

VALÓSZÍNŰ

Miért használnál több Availability Zone-t?

Az AZ-k fizikailag elkülönült infrastruktúrát adnak egy region belül. Multi-AZ deployment csökkenti az egy AZ hibájából eredő kiesést.

Az alkalmazásnak és az adatkomponenseknek is támogatniuk kell a failovert; önmagában a több példány nem elég, ha közös single point of failure marad.

Mit keres az interjúztató? High availability alapismeret.

ESSENTIAL

Hogyan kezelnéd a secretet és konfigurációt?

Secretet nem tennék repóba vagy image-be. Menedzselt secret store, IAM-alapú hozzáférés, rotáció és audit kell. A nem érzékeny konfigurációt környezetenként külön, validált módon adnám át.

Az alkalmazás startupkor fail fast módon ellenőrizheti a kötelező configot.

Mit keres az interjúztató? Biztonság és deployment hygiene.

Buktató A környezeti változó önmagában nem secret management stratégia; a tárolási és hozzáférési út számít.

VALÓSZÍNŰ

Mit jelent az Infrastructure as Code?

Az infrastruktúra deklaratív, verziózott és reviewzható definíciója. Reprodukálható környezetet, auditálhatóságot és automatizált deploymentet ad.

A module-határok, state kezelés, drift és change preview ugyanolyan fontosak, mint maga a template.

Mit keres az interjúztató? Repeatability és operations maturity.

Saját példád SAM/CloudFormation demo projektjeidet említheted.

BÓNUSZ

Hogyan gondolkodnál költségoptimalizálásról?

Először mérném, mi okozza a költséget, és üzleti egységhez kötném: például cost per shipment vagy per processed request. Right-sizing, au-

toscaling, storage lifecycle, query optimalizálás és architekturális egyszerűsítés jöhet szóba.

A költségoptimalizálás nem jelenthet vak reliability vagy fejlesztői produktivitás csökkentést.

Mit keres az interjúztató? Business-driven cloud szemlélet.

Saját példád Korábbi infrastruktúraköltség-csökkentési tapasztalatod jó példa lehet, ha a számot vállalod.

VALÓSZÍNŰ

Hogyan deployolnál kockázatsökkentve?

Automatizált pipeline, immutable artifact, environment promotion, smoke test és gyors roll-back. A változás kockázatától függően rolling, blue/green vagy canary deploymentet választanék.

Feature flag segíthet a deploy és release szétválasztásában, de a flag lifecycle-t menedzselni kell.

Mit keres az interjúztató? Release engineering és recoverability.

Saját példád Flagsmith, staging/production koordináció és rolling update tapasztalatod erős saját példa.

7. Git, CI/CD, minőség és observability

A hirdetés röviden említi a Git és CI/CD témát, de egy production full-stack szerepnél ezek gyakran a konkrét tapasztalatodon keresztül kerülnek elő.

ESSENTIAL

Mit tartalmazna egy jó CI pipeline?

Gyors visszajelzés: formatting/lint, type check, unit és releváns integration teszt, security/dependency check, build és artifact készítés. A drágább teszteket rétegezném, hogy a fejlesztő hamar kapjon eredményt.

Ugyanazt az artifactot promotálnám környezetek között, nem mindenhol újra buildelném.

Mit keres az interjúztató? Reprodukálható és gyors delivery folyamat.

Saját példád Jenkins, Docker és release coordination tapasztalatod.

VALÓSZÍNŰ

Trunk-based development vagy hosszú feature branchek?

Trunk-based rövid életű branchekkel, gyakori integrációval és feature flaggel csökkenti a merge kockázatot. Hosszú branch bizonyos izolált munkánál indokolt lehet, de driftet és késői integrációt okoz.

A csapat mérete, release szabályai és tesztelési kultúrája alapján választanék.

Mit keres az interjúztató? Collaboration és integration risk.

ESSENTIAL

Mit nézel code review-ban?

Helyesség, üzleti invariant, olvashatóság, API és komponenshatár, hibakezelés, tesztelhetőség, security, observability és migration kockázat. Nem csak style kérdéseket, amelyeket automation kezelhet.

A feedback legyen konkrét, indokolt és a kódra, ne a személyre irányuljon.

Mit keres az interjúztató? Senior együttműködés és minőségi szemlélet.

Saját példád Mentoring és csapaton belüli tudásmegosztás.

VALÓSZÍNŰ

Mi a feature flag előnye és kockázata?

Elválasztja a deployt a release-től, segíti a fokozatos rolloutot és a gyors kikapcsolást. Kockázata a kombinatorikus állapot, elavult flag és tesztelési komplexitás.

Minden flagnek legyen owner, cél, default, mérés és eltávolítási terve.

Mit keres az interjúztató? Production rollout tapasztalat.

Saját példád Flagsmith integráció és release management.

ESSENTIAL**Mi a structured logging?**

A log géppel feldolgozható mezőket tartalmaz: timestamp, level, service, environment, correlation ID, operation és releváns domain context. Ez jobb keresést, aggregációt és alertet tesz lehetővé, mint a szabad szöveg.

PII és secret nem kerülhet logba.

Mit keres az interjúztató? Diagnosztizálhatóság és privacy.

Saját példád Kibana és production troubleshooting.

VALÓSZÍNŰ**Mi a különbség metric, log és trace között?**

A metric aggregált numerikus idősor, jó trendhez és alerthez. A log részletes eseményrekord. A distributed trace egy request útját és span-jeit mutatja szolgáltatások között.

Együtt használva gyorsabban lehet a symptomtól a root cause-ig jutni.

Mit keres az interjúztató? Observability fogalmi tisztaság.

Saját példád Prometheus/Grafana, Sentry/Kibana mellett AWS CloudWatch/X-Ray tanulás.

BÓNUSZ**Mit jelent az SLO és error budget?**

Az SLO mérhető szolgáltatási cél, például a sikeres kérések vagy latency aránya. Az error budget azt fejezi ki, mennyi megbízhatósági eltérés fér bele a cél mellett.

Segít egyensúlyt teremteni feature delivery és reliability munka között.

Mit keres az interjúztató? Business és operations közös nyelve.

VALÓSZÍNŰ

Hogyan reagálnál production incidentre?

Először stabilizálnám a rendszert: rollback, feature disable, traffic reduction vagy failover. Közben egyértelmű kommunikáció és időbélyegzett eseménynapló kell.

Utána root cause és blameless postmortem, konkrét corrective action ownerrel és határidővel.

Mit keres az interjúztató? Nyomás alatti prioritás és tanuló kultúra.

Saját példád Release és monitoring felelősségedből hiteles példát adhatsz.

Buktató Az incident közben nem a tökéletes root cause elemzés az elsődleges, hanem az impact csökkentése.

8. LLM, MCP és AI agents

Ez a szerep megkülönböztető része. Valószínűleg nem kutatói mélységet várnak, hanem jó fogalmi modellt, biztonságos prototípust és üzleti use-case gondolkodást.

ESSENTIAL

Mi a különbség egy LLM feature és egy AI agent között?

Egy egyszerű LLM feature jellemzően egy vagy néhány model callból áll, például összefoglalás vagy klasszifikáció. Agent esetén a modell cél alapján több lépést tervezhet, toolokat választhat, eredményt értékelhet és iterálhat. Az agent nagyobb rugalmasságot ad, de növeli a latencyt, költséget, nondeterminizmust és biztonsági kockázatot.

Mit keres az interjúztató? Nem nevez-e minden LLM callt agentnek.

Saját példád Saját tanulási/automatizálási projektekből említhetsz workflow-t, ahol a modell strukturált eredményt ad vagy toolt hív.

ESSENTIAL

Mi az MCP?

A Model Context Protocol egy nyílt kliens-szerver protokoll, amely szabványosítja, hogyan ér el egy AI alkalmazás külső kontextust és képességeket. A server tipikusan tools, resources és prompts primitíveket tesz elérhetővé.

Az MCP nem maga az agent framework és nem dönti el, hogyan használja a host az LLM-et; a kapcsolódási és képességszerződést standardizálja.

Mit keres az interjúztató? Pontos, hype-mentes definíció.

Saját példád Példa: logisztikai rendszer MCP servere shipment lookup és pricing explanation toolokkal.

Buktató Ne mondd, hogy az MCP „adatbázis az LLM-nek” vagy teljes orchestration platform.

ESSENTIAL

Mi az MCP host, client és server szerepe?

A host az AI alkalmazás, amely kezeli a felhasználói élményt, engedélyeket és LLM integrációt. Minden serverhez külön client kapcsolatot hoz létre. A server fókuszált resources, prompts és tools képességeket kínál.

A szeparáció biztonsági és modularitási határt ad.

Mit keres az interjúztató? Érti-e a protokoll architektúráját.

VALÓSZÍNŰ

Mi a különbség MCP tool, resource és prompt között?

A tool végrehajtható művelet, például shipment lekérdezés vagy booking létrehozás. A resource kontextusadat, például szabályzat vagy rekord. A prompt újrahasználató interakciós sablon. A hozzáférési és kontrollmodell eltér: érzékeny toolnál különösen fontos a jóváhagyás és authorization.

Mit keres az interjúztató? Helyesen modellezi-e a capabilityt.

ESSENTIAL

Hogyan tennél biztonságossá egy AI tool callt?

A modellt nem tekinteném megbízható authorization komponensnek. A tool szerver validálja a bemenetet, ellenőrzi a felhasználói jogosultságot, korlátozza a scope-ot, timeoutot és rate limitet alkalmaz, és auditál.

Visszafordíthatatlan vagy nagy hatású művelethez explicit human confirmation és preview szükséges.

Mit keres az interjúztató? Security boundary és human-in-the-loop gondolkodás.

Saját példád Példa: árajánlat előkészíthető automatikusan, de végleges módosítás csak jóváhagyással.

Buktató A system prompt nem security boundary.

ESSENTIAL

Mi a prompt injection?

Olyan támadás vagy nem kívánt utasítás, amely a modell kontextusán keresztül próbálja felülmúlni a célokat vagy adatot exfiltrálni. Jöhet közvetlen user inputból vagy közvetetten dokumentumból, weboldalból, emailből.

Védekezés: adat és utasítás elkülönítése, allowlistelt toolok, least privilege, output validation, sandbox, confirmation és érzékeny adatok minimalizálása.

Mit keres az interjúztató? LLM security realitás.

Buktató Nincs egyetlen prompt, amely garantáltan megoldja a prompt injectiont.

ESSENTIAL

Mi a RAG?

Retrieval-Augmented Generation során a kérdéshez releváns külső tudást keresünk, és ezt adjuk a modell kontextusába. Jó változó vagy vállalatspecifikus információnál, mert nem kell minden tudást a modellparaméterekbe kódolni.

A minőséget a chunking, retrieval, metadata filtering, ranking, context construction és evaluation együtt határozza meg.

Mit keres az interjúztató? Nem keveri-e össze a finetuninggal.

Saját példád Logisztikai szabályzatok, ügyfélcontractok vagy belső knowledge base keresése.

Buktató A RAG nem garantál helyes választ; forrásminőség és grounding mérés kell.

VALÓSZÍNŰ

RAG vagy fine-tuning?

RAG friss, privát vagy forrásolható tudás bevonására jó. Fine-tuning inkább viselkedés, stílus vagy stabil feladatspecifikus minta javítására, megfelelő adatkészlettel.

Sok esetben prompt + RAG + tool call elég, és egyszerűbb, mint fine-tuning.

Mit keres az interjúztató? A problémát a legkisebb szükséges megoldással kezeli-e.

ESSENTIAL**Hogyan értékelnél egy LLM feature-t?**

Előre definiált feladatkészletet és üzleti metrikát használnék: pontosság, completeness, groundedness, invalid action rate, latency, cost és human acceptance. Automatikus eval mellett rendszeres emberi review is kell.

A production feedbacket verziózott prompt/model/config változásokhoz kötném.

Mit keres az interjúztató? Mérés és product thinking.

Saját példád Interjúfelkészítő vagy job-analyzer hobby projektben strukturált output és minőségi ellenőrzés példaként használható.

Buktató A „jól hangzik” nem evaluation.

VALÓSZÍNŰ**Hogyan kezelnéd a strukturált LLM outputot?**

Schema-constrained vagy structured outputot kérnék, majd runtime validálnám. Hibás output-nál kontrollált retry, repair vagy fallback jöhet szóba.

A downstream rendszer soha ne bízson vakon a modell JSON-jában.

Mit keres az interjúztató? Deterministic integration és runtime safety.

Saját példád A saját automatizálási projektjeidben JSON contract lehet releváns példa.

VALÓSZÍNŰ**Mit jelent az agent orchestration?**

Az agent lépéseinek, tool hívásainak, állapotának, retryainak, approval pontjainak és limit-

jeinek koordinálása. Lehet egyszerű state machine, workflow engine vagy agent framework. A legtöbb üzleti flow-nál a kontrollált workflow előnyösebb a teljesen nyitott autonómiánál.

Mit keres az interjúztató? Kontroll vs autonómia trade-off.

ESSENTIAL

Hogyan korlátoznád egy agent futását?

Maximum lépcsőszám, token- és költségbudget, timeout, tool allowlist, per-tool quota, circuit breaker és approval gate. Az állapotot és minden döntést auditálhatóan naplóznam.

Failure esetén legyen safe fallback és egyértelmű user feedback.

Mit keres az interjúztató? Production guardrails.

Buktató A végtelen agent loop valós költség- és üzemeltetési kockázat.

VALÓSZÍNŰ

Milyen logisztikai AI use case-t javasolnál?

Elsőként alacsony kockázatú, mérhető copilot funkciót: shipment exception összefoglalás, ügyfélemail draft, belső szabályzat keresés vagy pricing explanation. Ezeknél ember dönt, az AI gyorsít.

Csak később mennék autonóm módosítás felé, amikor az eval, permission és audit mechanizmusok bizonyítottak.

Mit keres az interjúztató? Üzleti érték és kockázat egyensúlya.

Saját példád Automation és business-oriented szemléletedet hangsúlyozhatod.

Buktató Ne azzal kezdj, hogy „AI mindent automatizál”.

BÓNUSZ

Mi a tool result poisoning kockázata?

Egy külső tool vagy resource rosszindulatú vagy félrevezető tartalmat adhat vissza, amely új utasításként befolyásolja a modellt. A tool outputot adatként kell kezelni, validálni, eredetét és trust leveljét nyomon követni.

Érzékeny műveletet nem szabad kizárólag egy nem megbízható tool szöveges eredményére alapozni.

Mit keres az interjúztató? Haladó agent security gondolkodás.

VALÓSZÍNŰ

Hogyan kezelné az LLM latencyt és költséget?

Kisebb modellt választanék, ha elég; rövidíteném a promptot és a contextet; cacheelném a stabil eredményt; párhuzamosítanám a független lépéseket; streaminggel javítanám a perceived latencyt.

A modellhívásokat üzleti értékhez és quality targethez mérném, nem csak tokenárhoz.

Mit keres az interjúztató? Pragmatikus production optimalizálás.

9. System design gyakorlatok

A megoldás fontos, de még fontosabb, hogy strukturáltan gondolkodj: requirements → constraints → high-level design → data → failure modes → trade-offs → rollout.

Válaszsablon

1. Tisztázó kérdések. 2. Funkcionális és nem funkcionális elvárások. 3. API és fő komponensek. 4. Adatmodell és consistency. 5. Skalázás és failure handling. 6. Security/observability. 7. Trade-off és fokozatos bevezetés.

Gyakorlat A - Shipment tracking platform

Feladat

Tervezz rendszert, amely shipment státuszokat fogad különböző partnerektől, normalizálja őket, és közel valós időben megjeleníti a webes felületen.

- Tisztázd a partnerek számát, event volumet, ordering igényt, maximum késést és adatmegőrzést.
- Ingress API vagy partner adapterek → durable queue/event bus → normalizer → shipment state store → notification/realtime gateway.

- Minden partner event kapjon idempotency key-t; out-of-order eseményhez sequence vagy timestamp szabály kell.
- A current state és az audit event history külön modellezhető.
- Frontend RESTtel tölti az aktuális állapotot, Web-Socket/SSE frissíti az új eseményeket.
- DLQ, replay, partnerenkénti rate limit és schema versioning szükséges.
- Metric: end-to-end event delay, invalid event rate, queue age, stale shipment count.

Erős trade-off mondat

A current state gyors lekérdezését és a teljes eseménytörténet auditálhatóságát külön optimalizálnám; nem kényszeríteném ugyanazt az adattárolási modellt mindkét célra.

Gyakorlat B - Dinamikus pricing workflow

Feladat

Tervezz belső alkalmazást, amely shipment adatokból és pricing rule-okból árajánlatot számol, emberi jóváhagyással, később AI-alapú javaslatokkal.

- A pricing engine legyen determinisztikus és verziózott; minden quote tárolja a használt rule verziót és input snapshotot.
- A UI draftot készít, backend validál és számol; optimistic concurrency védi a párhuzamos módosítást.
- Approval állapotgép: draft → calculated → pending approval → approved/rejected → expired.

- AI csak magyarázatot, anomaly jelzést vagy javasolt korrekciót adjon; a végleges árképzés maradjon auditálható.
- Legacy PHP pricing modul adapter mögé kerülhet; contract test és shadow comparison segítheti a migrációt.
- Metric: quote turnaround time, manual override rate, rule error rate, AI suggestion acceptance.

Erős trade-off mondat

Az AI-t nem tenném a determinisztikus árazási szabályok helyére. Először copilotként használnám, mert így az üzleti kockázat alacsony, az érték mégis mérhető.

Gyakorlat C - AI operations assistant MCP-vel

Feladat

Tervezz belső asszisztenst, amely képes shipmentet keresni, kivételes eseményeket összefoglalni és támogatási választ előkészíteni.

- Host: belső web app autentikált userrel. MCP serverek: shipment lookup, knowledge resources, customer communication draft.
- Read-only toolokkal indulnék. Write action külön server vagy külön privilege scope, explicit confirmationnel.
- A user identity és authorization minden tool call-ban propagálódik; a modell nem választ jogosultságot.
- A retrieval eredményhez forrás és timestamp tartozik; a válasz jelzi a bizonytalanságot.
- Evaluation: helyes shipment kiválasztás, grounded summary, forbidden action rate, latency, user acceptance.

- Audit: prompt/version, tool input-output, user approval és final action naplózva, érzékeny adatok maszkolva.

Erős trade-off mondat

Az agent rugalmasságát a tooloknál szűk, determinisztikus és jogosultság-ellenőrzött API-val ellensúlyoznám. A modell javasolhat, de a rendszer dönti el, mi hajtható végre.

System design gyors kérdések

ESSENTIAL

Mi az első kérdésed egy homályos system design feladatnál?

A felhasználó és az üzleti cél: ki használja, milyen döntést vagy folyamatot támogatunk, és mi számít sikernek. Ezután skála, latency, availability, consistency és compliance.

A requirements tisztázása nélkül a technológiai döntés csak találgatás.

Mit keres az interjúztató? Business-first gondolkodás.

ESSENTIAL

Hogyan azonosítod a single point of failure-t?

Végigkövetem a kritikus request vagy event útját, és minden komponensnél megkérdezem: mi történik, ha lassú, kiesik, duplikál, részben sikerül vagy elveszíti a kapcsolatot.

A redundancy mellett az automatikus failover és a recovery tesztelése is kell.

Mit keres az interjúztató? Failure-mode reasoning.

VALÓSZÍNŰ**Consistency vagy availability?**

Nem abszolút választásként kezelném. Üzleti műveletenként határoznám meg: például quote approvalnál erős consistency kellhet, tracking displaynél néhány másodperc eventual consistency elfogadható.

A döntést a hibás vagy késő adat üzleti következményéhez kötöm.

Mit keres az interjúztató? Trade-off konkrét domainhez kötése.

VALÓSZÍNŰ**Mikor bontanál microservice-ekre?**

Ha világos domainhatár, független skálázási vagy release igény, eltérő reliability/security profil vagy valódi csapattulajdonlás indokolja. Nem a kódméret önmagában.

Kezdetben moduláris monolith gyakran gyorsabb és egyszerűbb; később bizonyított határon lehet bontani.

Mit keres az interjúztató? Pragmatizmus és distributed-system cost felismerése.

Saját példád Monolith és microservice/realtime tapasztalat összevetése.

Buktató A microservice nem automatikus skálázhatósági megoldás.

ESSENTIAL**Hogyan tervezel migrationt?**

Kicsi, visszafordítható lépésekben. Adapter, dual run vagy shadow traffic, adat backfill, mérés, feature flag és rollback plan. Előre definiálnám a siker- és stop-kritériumokat.

Az üzleti folytonosság fontosabb, mint a technikai tisztaság gyors elérése.

Mit keres az interjúztató? Delivery risk és production safety.

Saját példád Legacy protokoll reverse engineering és fokozatos modernizáció.

10. Business és viselkedési kérdések

A Redspher business-oriented és pragmatikus hozzáállást kér. A STAR szerkezet mellett mindig mondd ki az üzleti hatást és a trade-offot.

ESSENTIAL

Mesélj egy end-to-end feature-ről.

Válassz olyan történetet, ahol requirement tisztázástól production monitoringig részt vettél. Mondd el a felhasználói problémát, a frontend/backend felelősséget, a döntést, a release-t és az eredményt.

Jó példa lehet GraphQL integráció, support chat modernizáció vagy egy teljes React feature release-e.

Mit keres az interjúztató? Ownership és eredmény, nem pusztán implementáció.

Saját példád Emeld ki, hogyan egyeztetted QA-val, business-szel és más technikai csapatokkal.

Buktató Ne töltsd a válasz nagy részét alacsony szintű kód részletekkel.

ESSENTIAL

Mondj példát pragmatikus technikai döntésre.

Mutasd be, hogy több opciót mérlegeltél, de a legegyszerűbb megoldást választottad, amely teljesítette az üzleti és operációs követelményt. Mondd ki azt is, mit nem építettél meg és miért.

Például fokozatos modernizáció, existing tooling használata vagy feature flag a nagy rewrite helyett.

Mit keres az interjúztató? Overengineering elkerülése.

ESSENTIAL

Hogyan kezeled a homályos üzleti igényt?

Példákkal és ellenpéldákkal tisztázom a desired behavior-t, az edge case-eket és a siker metrikáját. Rövid prototípus vagy UI flow gyakran gyorsabban tisztáz, mint hosszú specifikáció.

A feltételezéseket dokumentálom és visszaigazoltatom.

Mit keres az interjúztató? Stakeholder kommunikáció és requirement discovery.

Saját példád Figma/Storybook és product collaboration tapasztalat.

VALÓSZÍNŰ

Mit teszel, ha nem értesz egyet egy architekturális döntéssel?

Először megértem a döntés célját és constraintjeit. Adatokkal, konkrét kockázattal és alternatívával érvelek, lehetőleg kis spike vagy mérés alapján.

Ha a csapat döntött, elkötelezetten végrehajtom, hacsak nincs súlyos security vagy etikai kockázat.

Mit keres az interjúztató? Ego nélküli senior együttműködés.

ESSENTIAL**Mesélj production problémáról, amit megoldottál.**

STAR: impact → stabilizálás → diagnosztika → root cause → javítás → megelőzés. Emeld ki a monitoringot és a kommunikációt.

Választhatsz latency, package size anomaly, real-time connection vagy release problémát.

Mit keres az interjúztató? Incident handling és tanulás.

Saját példád Sentry/Kibana/Prometheus és automation példák.

Buktató Ne csak a technikai hőstettet mondd; mutasd be a csapat- és user impactot.

VALÓSZÍNŰ**Hogyan tanulsz új technológiát?**

Először a problémát és a mental modelt értem meg, majd kicsi működő prototípust készítek. Ezután official docs, failure case, security és operations oldal jön.

A tanulást akkor tekintem hasznosnak, ha el tudom magyarázni a trade-offot és tudom, mikor nem használnám.

Mit keres az interjúztató? Kíváncsiság és gyakorlatias tanulás.

Saját példád AWS demo projektek, interview trainer és AI automatizálás.

VALÓSZÍNŰ**Hogyan becsülsz meg egy feature-t?**

Felbontom user-visible slice-okra, azonosítom a bizonytalanságot és függőségeket, majd range-et adok feltételezésekkel. Ismeretlen kockázatra time-boxed spike jöhet.

A becslést frissítem, amikor új információ érkezik; nem kezelem ígéretként a korai számot.

Mit keres az interjúztató? Tervezés és transzparencia.

ESSENTIAL

Hogyan egyensúlyozod a gyorsaságot és a minőséget?

A kritikus invariant, security és recoverability nem opcionális. A scope és az absztrakció mélysége viszont csökkenthető. Kis vertical slice, feature flag és mérés segítségével gyorsan szállítok, majd visszacsatolás alapján bővítek.

A technical debt legyen tudatos, dokumentált és kockázathoz kötött.

Mit keres az interjúztató? Pragmatizmus minőségrombolás nélkül.

VALÓSZÍNŰ

Hogyan mentorálsz más fejlesztőt?

Nem csak megadom a választ, hanem segítek a probléma felbontásában és a döntési szempontokban. A feladat nehézségét fokozatosan emelem, és konkrét, gyors feedbacket adok.

Pairing, code review, dokumentáció és rövid belső előadás különböző helyzetekben működik.

Mit keres az interjúztató? Team impact és kommunikáció.

Saját példád Új kollégák mentorálása és belső technikai előadások.

BÓNUSZ**Miért logisztika?**

A logisztika valós idő-, adat- és optimalizációs problémákat kapcsol össze közvetlen üzleti hatással. Érdekes, hogy egy technikai javítás mérhetően csökkentheti a késést, manuális munkát vagy árazási hibát.

A szerepben különösen vonzó, hogy a klasszikus full-stack fejlesztés AI-driven workflow-k felé fejlődik.

Mit keres az interjúztató? Őszinte motiváció, nem iparági közhely.

11. Saját történetbank

Az alábbi témákat készítsd elő 2 perces STAR válaszként. Egy történet több kérdésre is újra-hasznosítható, de mindig az adott kérdésre fókuszálj.

1. Skálázható realtime backend

- Situation: legacy support chat vagy event broadcaster korlátai.
- Task: skálázhatóbb, biztonságosabb Node.js/WebSocket megoldás.
- Action: protocol feltérképezés, JWT, Redis pub/sub, clustering, rolling update, monitoring.
- Result: jobb latency/throughput és alacsonyabb működési költség - csak vállalható számokkal.
- Használható: Node event loop, scaling, migration, resilience, incident, architecture.

Készülj számmal

Idő, latency, throughput, payload, költség vagy hibaarány közül legalább egy mérőszám legyen a történetben. A mérőszámot és mérési módját tudd megmagyarázni.

2. React/TypeScript end-to-end feature ownership

- Situation: nagy SPA üzleti feature vagy integráció.
- Task: requirementtől production release-ig delivery.

- Action: komponens/API tervezés, state, QA/business együttműködés, feature flag, monitoring.
- Result: gyorsabb vagy stabilabb user flow, mérhető payload vagy UX javulás.
- Használható: React architecture, stakeholder, CI/CD, pragmatic decision, ownership.

Készülj számmal

Idő, latency, throughput, payload, költség vagy hibaarány közül legalább egy mérőszám legyen a történetben. A mérőszámot és mérési módját tudd megmagyarázni.

3. Automatizálás és operációs hatás

- Situation: manuális report/dokumentáció vagy asset anomaly.
- Task: ismétlődő munka és rejtett hiba csökkentése.
- Action: Node.js automation, REST API integráció, ellenőrzés és visszajelzés.
- Result: időmegtakarítás, kisebb production package vagy jobb láthatóság.
- Használható: innovation, business value, quality, proactive behavior, AI/automation mindset.

Készülj számmal

Idő, latency, throughput, payload, költség vagy hibaarány közül legalább egy mérőszám legyen a történetben. A mérőszámot és mérési módját tudd megmagyarázni.

4. Release és production monitoring

- Situation: több környezet, QA és üzleti egyeztetés.
- Task: biztonságos release és gyors hibaészlelés.
- Action: staging validation, rollout coordination, dashboards, feature flag, rollback readiness.
- Result: kiszámíthatóbb release és gyorsabb diagnosztika.
- Használható: CI/CD, incident, communication, ownership, observability.

Készülj számmal

Idő, latency, throughput, payload, költség vagy hibaarány közül legalább egy mérőszám legyen a történetben. A mérőszámot és mérési módját tudd megmagyarázni.

5. Új terület gyors megtanulása

- Situation: AWS vagy AI/agent technológiák bővítése.
- Task: ne csak elméleti, hanem használható tudás.
- Action: official docs, cert training, kis serverless projektek, strukturált outputot használó hobby alkalmazások.
- Result: működő prototípusok és tudatosabb architecture trade-offok.
- Használható: curiosity, MCP/AI interest, AWS, self-development.

Készülj számmal

Idő, latency, throughput, payload, költség vagy hibaarány közül legalább egy mérőszám legyen

a történetben. A mérőszámot és mérési módját tudd megmagyarázni.

12. Gyors kérdések - egymondatos válaszok

Ezekkel az interjúztató gyakran ellenőrzi a fogalmi tisztaságot. Először mondd a lényegét, utána kérésre részletezz.

VALÓSZÍNŰ

Authentication vs authorization

Authentication: ki vagy? Authorization: mit tehetsz meg?

VALÓSZÍNŰ

Horizontal vs vertical scaling

Horizontal: több instance; vertical: nagyobb erőforrás egy instance-nak.

VALÓSZÍNŰ

Stateless service

A request feldolgozásához szükséges tartós session state nincs egy konkrét instance memóriájához kötve.

VALÓSZÍNŰ

Cache invalidation

A cache értékét frissíteni vagy lejáratni kell, amikor a forrásadat változik; a consistency igény dönti el a stratégiát.

VALÓSZÍNŰ**Circuit breaker**

Ismétlődő dependency hibánál ideiglenesen leállítja a hívásokat, hogy gyorsan fail-eljen és időt adjon a helyreállásra.

VALÓSZÍNŰ**Timeout**

Minden hálózati műveletnek legyen felső időkorlátja; különben erőforrások maradhatnak foglalta korlátlan ideig.

VALÓSZÍNŰ**Optimistic vs pessimistic locking**

Optimistic konfliktust updatekor észlel; pessimistic előre lockol. Ritka ütközésnél általában optimistic jobb.

VALÓSZÍNŰ**Debounce vs throttle**

Debounce a csend után fut; throttle egy időablakban korlátozza a futások számát.

VALÓSZÍNŰ**CORS**

Böngésző által kikényszerített cross-origin hozzáférési szabály, amelyet a szerver response headerei engedélyeznek.

VALÓSZÍNŰ**CSRF**

A böngésző automatikusan küldött credential-jeit használja ki egy másik oldalról indított kérés; SameSite és CSRF token gyakori védelem.

VALÓSZÍNŰ **XSS**

Nem megbízható tartalom scriptként fut a böngészőben; output encoding, safe templating és CSP segít.

VALÓSZÍNŰ **ACID**

Atomicity, Consistency, Isolation, Durability - tranzakciós tulajdonságok.

VALÓSZÍNŰ **Eventual consistency**

Az adatpéldányok nem azonnal, de idővel konvergálnak; az átmeneti eltérést a domainnek kezelnie kell.

VALÓSZÍNŰ **Dead-letter queue**

A többször sikertelen message elkülönített helye elemzéshez és kontrollált újrafeldolgozáshoz.

VALÓSZÍNŰ **Correlation ID**

Egy azonosító, amellyel ugyanazon request vagy workflow eseményei több szolgáltatáson át összeköthetők.

VALÓSZÍNŰ**Canary deployment**

A változást kis traffic részre engedjük ki, mérjük, majd fokozatosan növeljük.

VALÓSZÍNŰ**Blue/green deployment**

Két teljes környezet között váltunk, ami gyors rollback ad, de több erőforrást igényel.

VALÓSZÍNŰ**RPO vs RTO**

RPO: mennyi adatvesztés fér bele; RTO: mennyi idő alatt kell helyreállni.

VALÓSZÍNŰ**SSE vs WebSocket**

SSE szerver → kliens egyirányú stream HTTP-n;
WebSocket kétirányú, tartós kapcsolat.

VALÓSZÍNŰ**MCP röviden**

Szabványos protokoll, amellyel AI hostok tools, resources és prompts képességeket érnek el servereken keresztül.

13. 30 perces próbainterjú

Állíts időzítőt. A technikai válaszok 60-90 másodpercesek, a történetek 2 percesek legyenek.

0-5 perc - bemutatkozás

- Tell me about yourself.
- Why are you interested in Redspher and this role?
- Which part of the stack is your strongest, and where are you growing?

5-15 perc - core stack

- Explain a React performance problem you diagnosed.
- When should you not use `useEffect`?
- How do you validate external data in TypeScript?
- How does Node.js handle many concurrent connections?
- How would you design consistent API errors and retries?

15-23 perc - architecture

- Design a shipment tracking flow with near-real-time updates.
- Where would you use a queue, and how would you handle duplicates?
- What would you monitor and alert on?
- How would you deploy a risky change safely?

23-28 perc - AI

- What is MCP, and how is it different from an agent framework?
- Design a safe AI tool that can look up shipment data.
- How would you evaluate whether the feature is useful?

28-30 perc - te kérdezel

- What are the main business workflows the team owns today?
- How is the responsibility split between frontend, backend and platform work?
- Which AI use cases are already in production, and which are still exploratory?
- How do you evaluate prototypes before turning them into product features?
- What would success look like in the first three to six months?

14. Utolsó napi cheat sheet

Az interjú előtt ezt a részt olvasd át. Ne próbálj már új témát megtanulni.

Mondd ki ezeket

Requirements, trade-off, failure mode, idempotency, validation, observability, rollback, least privilege, business impact, measured result.

React

- Render legyen pure; effect csak external system szinkronizációra.
- State colocation; globális state csak valós cross-feature igénynél.
- Mérés előtt ne memoizálj vakon.
- Loading, empty, error és stale response legyen explicit.

TypeScript

- `unknown` + narrowing a határokon; `any` csak kontrollált escape hatch.
- Runtime inputot schema validál.
- Discriminated union modellezi az állapotokat; `never` ellenőrzi a teljességet.
- Generics kapcsolatot őriz, nem csak „rugalmasabbá tesz”.

Node/API

- CPU munka ne blokkolja az event loopot.
- Timeout + kontrollált retry + idempotencia.
- Vékony route handler, tiszta use-case és infras-truktúrahatár.
- Strukturált hiba és correlation ID.

AWS

- Workload alapján Lambda vagy container.
- Least privilege, managed secret, IaC.
- Multi-AZ, queue buffering, DLQ, monitoring és tested recovery.
- Well-Architected trade-off: reliability, security, performance, cost, operations, sustainability.

AI/MCP

- MCP: host-client-server, JSON-RPC alapú tools/resources/prompts szerződés.
- A modell nem authorization boundary.
- Read-only és copilot use case-szel indulj.
- Schema validation, budget, timeout, audit, human approval.
- Eval nélkül nincs production-ready AI feature.

A legfontosabb interjúmondat

I would start with the business requirement and the failure modes, then choose the simplest architecture that meets the reliability and delivery needs.

15. Források és hatókör

Az anyag a mellékelt Redspher álláshirdetésből és stabil, hivatalos technikai dokumentációk alapelveiből készült.

- Redspher Full Stack Developer job description - Contern, Luxembourg.
- TypeScript Handbook - generics, narrowing, unknown, never és strict type checking.
- React official documentation - rendering, state, effects és performance szemlélet.
- AWS Well-Architected Framework - operational excellence, security, reliability, performance efficiency, cost optimization és sustainability.
- Model Context Protocol official specification és architecture overview - host/client/server, tools, resources, prompts, transport és security alapelvek.

Fontos

A hirdetés nem ad konkrét információt a backend frameworkről, adatbázisról, AWS szolgáltatásokról, tesztstackről vagy az AI platformról. Az ezekre vonatkozó részek valószínű interjú témák és felkészítő példák, nem a Redspher belső architektúrájára vonatkozó állítások.